スポットインスタンス上の短時間ワークロードにおける実行時刻と分割粒度を考慮した中断対策

那須 渚

2026 年度 修士論文

同志社大学大学院 理工学研究科

概要

スポットインスタンスはクラウド利用コストを大幅に削減できる一方，提供側の都合により中断される可能性がある．CI/CD のビルドやテストのような短時間ワークロードでは，中断が発生するとジョブ全体の再実行が必要となり，実行時間と利用コストの双方が増加する．

本論文では，短時間ワークロードにおけるスポットインスタンスの信頼性を，起動可能性ではなく**期限条件付き完了可能性** $F_{\mathrm{comp}}(d)$ として再定義する．$F_{\mathrm{comp}}(d)$ は，起動に成功したインスタンスがワークロード期限 d まで中断されずに利用可能であり続ける割合である．

AWS の 4 リージョン・3 インスタンスタイプを対象として 1 週間（$N_{\mathrm{req}} = 2{,}016$）の実測を行い，$F_{\mathrm{comp}}(d)$ がローカル時間帯によって大きく変動することを示した．計算指向インスタンスでは，Night 期間の $F_{\mathrm{comp}}(60) = 0.98$ に対し Morning 期間では 0.69 まで低下した．この結果は，**実行時刻**が運用上意味のあるスケジューリング要因であることを示す．

しかし時間帯の選択は，最大で 24 時間の待機を要するため，即時性が求められる CI/CD では適用範囲が限られる．そこで本論文ではさらに，取得済みデータの再分析を通じて**分割粒度**という第二の運用変数を導入する．

再分析の結果，中断ハザード率は経過時間によらずほぼ一定（0.054–0.130 %/分）であり，中断が近似的にメモリレスな事象として振る舞うことが示された．この性質のもとでは，ジョブを k 分割することで被中断確率と中断時損失量が同時に低下する．一方で分割には起動およびチェックポイントのコストが伴うため，総無駄時間は分割数に対して U 字型となり，**最適分割数 k^* が存在する**．

さらに感度分析により，k^* がチェックポイントコスト c に極めて敏感であることを明らかにした．$c = 0$ では $k^* = 6$ であるが，$c = 5$ 秒で $k^* = 2$ に低下し，$c \geq 60$ 秒では分割そのものが無効となる．c は未実測の量であり，提案手法が成立するか否かは c の値に依存する．

以上を踏まえ，本論文では c の実測を最優先とする実験計画を提示する．

目次

概要 1

第 1 章 序論 5
1.1 研究背景 5
1.2 問題意識 5
1.3 本研究の問い 6
1.4 本論文の貢献 6
1.5 本論文の構成 7

第 2 章 関連研究 8
2.1 スポットインスタンスの可用性と時間的変動 8
2.2 スポットインスタンスのデータセットとワークロード指向の利用 8
2.3 チェックポイントと分割実行 9
2.4 本研究の位置づけ 9

第 3 章 期限条件付き完了可能性 10
3.1 問題の定義 10
3.2 目的 10
3.3 アプローチ 10
3.4 評価指標の定義 11

第 4 章 実測評価 13
4.1 実験方法 13
4.2 全体結果 14
4.3 時間帯が完了可能性に与える影響 15
4.4 考察 16

第 5 章 分割粒度を考慮した実行モデル 18
5.1 本章のねらい 18
5.2 中断の近似的メモリレス性 18
5.3 分割がもたらす二つの効果 19
5.4 二変数を統合したコストモデル 20

5.5 最適分割数の存在 21
5.6 チェックポイントコストに対する感度 22
5.7 実行時刻との併用 23
5.8 比較対象とする戦略 24
5.9 モデルの未確定要素と検証計画 24
5.10 補足：リージョン選択について 28

第 6 章 結論 30
6.1 本論文のまとめ 30
6.2 本研究の意義 31
6.3 今後の課題 31

謝辞 32

参考文献 33

参考文献 33

図目次

3.1　提案する測定フレームワーク　. 11

4.1　ローカル時間帯，リージョン，インスタンスタイプ別の $F_{\mathrm{comp}}(60)$。注釈は値とサンプル数（$n_{\mathrm{comp}}/n_{\mathrm{succ}}$）を示す。. 14
4.2　時間帯間の変動が最大であった 4 つのリージョン・インスタンスタイプの組み合わせにおける $F_{\mathrm{comp}}(d)$。線はローカル時間帯を表し，横軸は対数スケールである。. 15

5.1　経過時間別の中断ハザード率。単調増加の傾向は見られず，0.054–0.130 %/分の範囲で変動する。. 19
5.2　分割数に対する総無駄時間の U 字カーブ。各曲線はチェックポイントコスト c の値に対応し，二重丸は各曲線の最小点 k^{*} を示す。c の増加とともに k^{*} は左へ移動する。. 23
5.3　ローカル時間帯と分割数の組み合わせによる総無駄時間。色が濃いほど無駄時間が大きい。. 24

表目次

1.1 本論文の構成 7

2.1 関連研究と本研究の位置づけ 9

4.1 評価対象インスタンスタイプの仕様。観測スポット価格は観測期間中（2026 年 5 月 19 日 –25 日）に 4 リージョンで記録された平均時間単価（USD/h）である。 13
4.2 ローカル時間帯の区分 14

5.1 経過時間別の中断ハザード率（$N_{\mathrm{succ}} = 1{,}950$，中断 117 件） 19
5.2 断片長 T に対する完了確率と中断時平均損失（全データをプール） 20
5.3 ジョブの特徴量 20
5.4 分割数 k に対する総無駄時間（$W_j = 60$ 分，$c = 0$，Wilson LCB） 22
5.5 チェックポイントコスト c に対する最適分割数 k^* の感度。各セルは総無駄時間（分）を表し，太字は各行の最小値を示す。 22
5.6 ローカル時間帯と分割数の組み合わせによる総無駄時間（分，$c = 0$，Wilson LCB） 23
5.7 比較する戦略 24
5.8 実験の目的と優先順位 25
5.9 対象ワークロードの候補 25
5.10 実験の規模とコスト（c6a.large を想定） 27
5.11 既存実装からの変更点 27
5.12 チェックポイントコストの実測値に応じた結論 28
5.13 想定されるリスクと対策 28
5.14 c6a.large のリージョン別観測スポット価格 29

第 1 章

序論

1.1 研究背景

近年，クラウドコンピューティングは企業，研究機関，個人開発者によって広く利用されている．クラウド利用の規模が拡大し，高性能な計算資源への需要が増加するにつれて，その利用コストは多くの利用者にとって重要な課題となっている．

コストを削減する手法の一つに，スポットインスタンスの利用がある．スポットインスタンスは，クラウドプロバイダが余剰計算資源をオンデマンドインスタンスより低価格で提供する料金モデルである [1]．オンデマンドインスタンスと比較して，数十パーセントから最大約 90% の割引価格で利用できる [2]．そのため，機械学習やバッチ処理のような大規模計算ワークロードだけでなく，CI/CD における自動ビルドやテストのように一時的に計算資源を必要とするワークロードにも利用されている [3]．

しかしスポットインスタンスは余剰容量に基づいて提供されるため，クラウド需要が増加した場合や利用可能容量が減少した場合に中断されうる．この中断を interruption event と呼ぶ．ワークロードの実行中に interruption event が発生すると，実行中の処理が完了前に停止する．特にバッチ処理，機械学習，CI/CD のビルドやテストが中断された場合，ワークロード全体の再実行が必要となり，実行時間と利用コストの双方が増加する．

1.2 問題意識

既存のアプローチとしては，価格変動を考慮した入札戦略や，Spot Placement Score（SPS）のようなプロバイダ固有の可用性指標に基づく選択が挙げられる [4, 5]．SPS は，スポットリクエストが成立する可能性を推定する上で有用である．

しかし，CI/CD パイプラインやバッチ処理ジョブのような短時間ワークロードの利用者にとって，より実用的な問いは**スポットインスタンスを起動できるかどうか**だけではなく，**ワークロードが完了するまで利用し続けられるかどうか**である．

起動に成功したとしても，ワークロード完了前に中断されれば，短時間ワークロードは失敗する．すなわち**起動時点の可用性**と**起動後の継続性**は異なる問題であり，既存指標は前者のみを扱っている．

1.3　本研究の問い

本論文を貫く問いは一つである．

中断によって失われる時間を最小化するには，ジョブをどう実行すればよいか．

この問いに答えるには，まず「失われる時間」を定量化する必要がある．

中断されるとジョブは再実行される．したがって利用者が支払う総時間は，本来必要な計算時間 W に加え，中断された試行で費やされた時間が上乗せされる．この上乗せ分を**無駄時間**と呼び，本論文の最小化対象とする．

無駄時間は二つの要素の積で決まる．

$$\text{無駄時間} \approx \underbrace{\text{やり直す回数}}_{\text{中断されにくさで決まる}} \times \underbrace{\text{1 回あたりの損失}}_{\text{1 回の実行の長さで決まる}} \tag{1.1}$$

この分解が本論文の構成を決定する．二つの要素はそれぞれ異なる運用変数によって制御できるからである．

■第一の運用変数：実行時刻 t（第 3–4 章）． 式 (1.1) の第一要素，すなわち**やり直す回数**を減らす．中断されにくい時間帯に実行すれば，再実行の回数が減る．そのためにはまず「ある時間帯にどれだけ中断されにくいか」を測る指標が必要であり，これを期限条件付き完了可能性 $F_{\mathrm{comp}}(d)$ として定義する．

■第二の運用変数：分割粒度 k（第 5 章）． 式 (1.1) の第二要素，すなわち **1 回あたりの損失**を減らす．ジョブを k 個の断片に分割すれば，中断されても失われるのは 1 断片分で済む．

二つの変数は式 (1.1) の異なる因子に作用するため，**独立に効き，併用できる**．また実行時刻の選択が待機を要するのに対し，分割は待機を必要としない．両者は代替関係にあると同時に補完関係にもある．

本論文は，この二変数を統一的に扱うコストモデルを構築し，実測データに基づいて最適な (t, k) を求める問題として定式化する．

1.4　本論文の貢献

本論文の貢献は以下の通りである．

1. 中断による無駄時間を最小化する問題を，**実行時刻 t と分割粒度 k の二変数最適化**として定式化した（第 3 章，第 5 章）．
2. その第一要素を測る指標として**期限条件付き完了可能性** $F_{\mathrm{comp}}(d)$ を定義し，起動成功と起動後の中断挙動を分離して記録する測定フレームワークを実装した（第 3 章）．
3. 4 リージョン・3 インスタンスタイプ・1 週間の実測により，$F_{\mathrm{comp}}(d)$ がローカル時間帯によって大きく変動することを示し，実行時刻が有効な運用変数であることを実証した

（第 4 章）.

4. 同一データの再分析により中断の近似的メモリレス性を示し，これに基づいて**分割粒度**を第二の運用変数として導入した（第 5 章）.
5. 二変数を統合したコストモデルを構築し，**最適分割数 k^* が存在する**こと，および k^* が**チェックポイントコストに支配される**ことを定量的に示した（第 5 章）.
6. モデルの未確定要素を同定し，それを最短で解消する検証計画を示した（第 5 章 §5.9）.

1.5 本論文の構成

本論文の構成と，各章が式 (1.1) のどの要素を扱うかを表 1.1 に示す.

表 1.1　本論文の構成

章	扱う要素	内容
第 3 章	やり直す回数	指標 $F_{\mathrm{comp}}(d)$ の定義と測定フレームワーク
第 4 章	やり直す回数	実測による時間帯依存性の実証と，実行時刻の適用限界
第 5 章	**両方**	1 回あたりの損失を減らす分割粒度の導入と，二変数を統合したモデルの構築・検証計画
第 6 章	—	結論

第 2 章

関連研究

2.1 スポットインスタンスの可用性と時間的変動

スポットインスタンスの可用性および信頼性については，これまでに複数の研究が行われている．Lin ら [2] は可用性を向上させる手法を調査し，チェックポイント，マイグレーション，レプリケーション，スケジューリングといったアプローチを整理している．これらの研究は，スポットインスタンスを実用的に利用するうえで中断リスクが中心的課題であることを示している．

SPS は AWS が提供するプロバイダ固有の指標であり，インスタンスタイプ，リージョン，必要容量などの条件下でスポットリクエストが成立する可能性を推定するために用いられる [7]．Katayama ら [5] はマイグレーション先の選択に SPS を用い，起動時点の可用性を向上させられることを示した．

Kim ら [6] は，SPS が時間帯や曜日に応じて周期的に変動し，survival time と一定の関係を持つことを示している．ただし同研究では，時間的変動は主にスポット市場の特性として扱われており，ワークロードの期限に基づく観点からは分析されていない．

本研究は，時間帯を**短時間ワークロードの実行タイミングを決定する運用上の変数**として扱う点で，これらと異なる．

2.2 スポットインスタンスのデータセットとワークロード指向の利用

Lee ら [8] は，SPS，スポット価格，中断率などを継続的に収集・保存するアーカイブサービス SpotLake を提案した．同研究では，GPU インスタンスや大規模インスタンスタイプで可用性が低くなる傾向があること，可用性がリージョンによって異なることを報告している．

Garg ら [4] は AWS Spot Instances に対する最適化された入札戦略を検討し，Ramesh と Lennon [3] は DevOps クラウドワークフローにおけるリザーブドインスタンスとスポットインスタンスの影響を評価している．これらはスポットインスタンスが CI/CD や DevOps に関連することを示すが，時間帯と短時間ジョブの完了可能性の関係は十分に検討されていない．

2.3　チェックポイントと分割実行

中断対策としてのチェックポイントは古くから研究されている [2]．機械学習の分野ではエポック単位でのモデル保存が一般的であり，CI/CD の分野でもテストのシャード分割やビルドキャッシュといった形で事実上の分割実行が広く行われている．

ただし既存研究の多くは，チェックポイントを「中断に備えて取るもの」として定性的に扱っており，**どの粒度で分割すべきか**という設計問題として定量的に扱った例は限られている．本研究は，分割粒度を最適化対象とし，その最適値がチェックポイントコストによって決まることを示す点で新しい．

2.4　本研究の位置づけ

既存研究に対する本研究の位置づけを表 2.1 に示す．

表 2.1　関連研究と本研究の位置づけ

	着目する側面	**運用変数**
SPS [7]，Katayama ら [5]	起動時点の可用性	リージョン，AZ
Kim ら [6]	市場の時間的変動，survival time	（分析対象であり運用変数ではない）
Lee ら [8]	インフラレベルの可用性データ	リージョン，インスタンスタイプ
本研究	**期限条件付き完了可能性**	**実行時刻，分割粒度**

第 3 章

期限条件付き完了可能性

3.1 問題の定義

SPS のような既存指標は，スポットリクエストが成立するかどうかを特徴づける．しかし，起動に成功したインスタンスがワークロード完了まで利用可能であり続けるかどうかは示さない．

CI/CD パイプラインやバッチジョブのような短時間ワークロードにとって，この二つは異なる関心事である．起動に成功しても完了前に中断されれば，ジョブ全体の再実行が必要となり，実行時間とコストの双方が増加する．したがって利用者には，リクエスト成立可能性ではなく**期限指向のスポット信頼性**が必要である．

さらに，多くの短時間ワークロードは実行時刻を柔軟に調整できる．したがって時間帯は中断リスクを低減するための実行可能な要因となりうる．しかし既存指標は，この時間帯の影響を期限指向の形で提示しない．

SPS と $F_{\mathrm{comp}}(d)$ は，スポット信頼性の異なる側面を捉える**相補的な指標**である．SPS は起動時点の配置可能性を，$F_{\mathrm{comp}}(d)$ は起動後のワークロード固有期限に対する継続性を扱う．

3.2 目的

本章の目的は，起動に成功したスポットインスタンスが 5，10，30，60 分というワークロード期限まで中断されずに利用可能であり続ける確率を測定し，それが時間帯，リージョン，インスタンスタイプによってどのように異なるかを明らかにすることである．

3.3 アプローチ

本研究では実際のアプリケーションレベルのワークロードは実行しない．代わりに，指定期限まで中断されずに利用可能であることを**ワークロード完了可能性の proxy** として用いる．すなわち，起動に成功したインスタンスが期限 d より前に中断されなかった場合，観測条件下においてその実行時間要件を持つ短時間ワークロードは完了可能とみなす．

図 3.1 に提案する測定フレームワークを示す．本フレームワークでは，スケジューラを用いてスポットインスタンスの起動リクエストを定期的に送信し，起動結果，稼働状態，中断状態を収

集する．

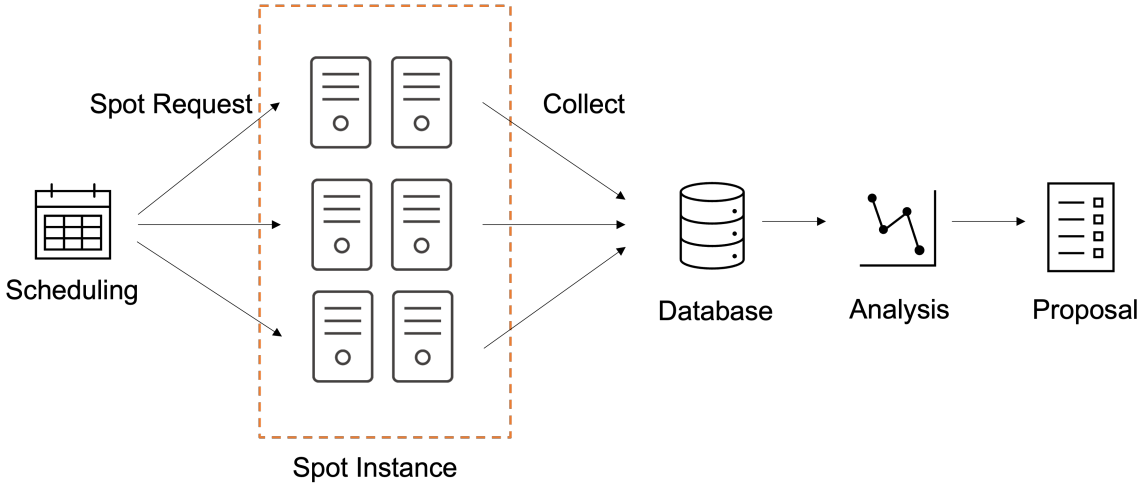


図 3.1　提案する測定フレームワーク

スケジューラは対象とする各インスタンスタイプに対して定期的に起動リクエストを送信する．起動に成功した各インスタンスは 1 時間の固定観測期間にわたって監視される．この期間中，システムはリクエストが成立したか，インスタンスがどの程度の時間利用可能であったか，予定終了時刻より前に interruption event が発生したかを記録する．観測期間内に中断が発生しなかった場合，インスタンスはシステムによって終了される．

3.4　評価指標の定義

N_{req} をスポットインスタンスの起動リクエスト数，N_{succ} を起動に成功したインスタンス数，$N_{\mathrm{int}}(d)$ をワークロード期限 d より前に中断された起動成功インスタンス数とする．

起動成功率を次のように定義する．

$$R_{\mathrm{launch}} = \frac{N_{\mathrm{succ}}}{N_{\mathrm{req}}} \tag{3.1}$$

期限前中断率を次のように定義する．

$$R_{\mathrm{int}}(d) = \frac{N_{\mathrm{int}}(d)}{N_{\mathrm{succ}}} \tag{3.2}$$

ここで d はワークロード期限である．本研究では $d = 5, 10, 30, 60$ 分を用いる．さらに**期限条件付き完了可能性**を次のように定義する．

$$\boxed{F_{\mathrm{comp}}(d) = 1 - R_{\mathrm{int}}(d)} \tag{3.3}$$

この指標は，起動に成功したインスタンスのうち期限 d まで利用可能な状態を維持した割合を表す．ただしこれは実際のアプリケーションレベルの成功率ではなく，**インスタンスの生存に基づく proxy** として解釈すべきである．

式 (3.1) と式 (3.2) を分離して定義することにより，起動時点の容量不足に起因する失敗と，起動後に発生する中断とを区別して分析できる．

第 4 章

実測評価

4.1　実験方法

提案フレームワークを AWS 上に実装した．スケジューラ駆動の Lambda 関数が定期的にスポットインスタンスを起動し，起動結果，稼働状態，中断イベントをデータベースに記録する．

■インスタンスタイプ． 異なるリソース特性を持つインスタンスタイプ間で中断傾向を比較するため，3 種類を代表例として選定した [9, 10]．仕様を表 4.1 に示す．

表 4.1　評価対象インスタンスタイプの仕様。観測スポット価格は観測期間中（2026 年 5 月 19 日 –25 日）に 4 リージョンで記録された平均時間単価（USD/h）である。

	c6a.large	**g4dn.xlarge**	**t3.micro**
分類	計算最適化	GPU 搭載	バースト可能汎用
プロセッサ	AMD EPYC (3rd gen)	Intel Xeon (Cascade Lake)	Intel Xeon (burstable)
vCPU	2	4	2 (burstable)
メモリ	4 GiB	16 GiB	1 GiB
GPU	—	1× NVIDIA T4 (16 GiB)	—
ローカルストレージ	EBS のみ	125 GB NVMe SSD	EBS のみ
観測スポット価格	0.038	0.277	0.005

■リージョンと時間帯． 対象リージョンは N. Virginia (us-east-1)，Tokyo (ap-northeast-1)，Ireland (eu-west-1)，Singapore (ap-southeast-1) の 4 つである．各リージョンにおいて，各インスタンスタイプにつき 1 時間に 1 台を起動する．

時間帯の傾向を分析する際，タイムスタンプは各リージョンのローカル時刻に正規化する．これにより，各リージョンのローカルな需要サイクルに対して中断傾向を比較できる．ローカル 1 日を 6 時間ずつの 4 区間に分割する．

スポットリクエストでは最大スポット価格を指定していない．したがって本実験は，AWS のデフォルトのスポット価格設定における挙動を観測している．

表 4.2　ローカル時間帯の区分

区分	ローカル時刻
Night	00:00–05:59
Morning	06:00–11:59
Afternoon	12:00–17:59
Evening	18:00–23:59

■観測期間． 実験は 2026 年 5 月 19 日から 5 月 25 日まで実施した．

■中断の判定． スポット中断と通常のシステム終了を区別するため，測定システムによって終了されることなく予定された 1 時間の観測終了前に利用不能となったインスタンスを中断候補として扱う．

4.2　全体結果

観測期間中, 4 リージョン･3 インスタンスタイプにわたり合計 $N_{\mathrm{req}} = 2{,}016$ 件の起動リクエストを送信した．このうち $N_{\mathrm{succ}} = 1{,}950$ 件が正常に起動し，全体の起動成功率は $R_{\mathrm{launch}} = 96.7\%$ であった．起動に成功したインスタンスのうち 117 件（6.0%）が 1 時間の観測期間終了前に中断された．

図 4.1 に，ローカル時間帯，リージョン，インスタンスタイプごとに集計した $F_{\mathrm{comp}}(60)$ を示す．

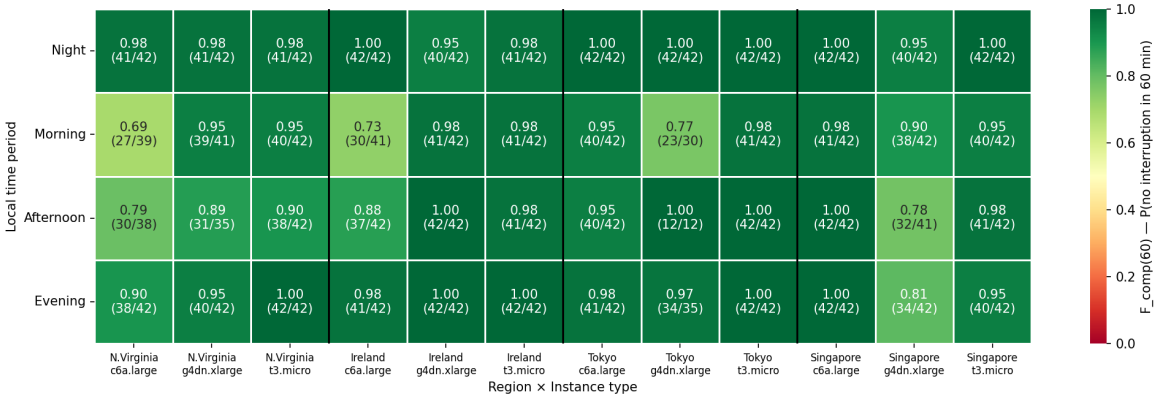


図 4.1　ローカル時間帯，リージョン，インスタンスタイプ別の $F_{\mathrm{comp}}(60)$。注釈は値とサンプル数（$n_{\mathrm{comp}}/n_{\mathrm{succ}}$）を示す。

t3.micro はすべてのリージョンおよび時間帯において $F_{\mathrm{comp}}(60) \geq 0.95$ を達成しており，軽量インスタンスは実行タイミングにかかわらず中断に対して比較的頑健であることを示している．一方 c6a.large および g4dn.xlarge では時間帯による変動が大きく，値はおおよそ 0.65 から 1.00 の範囲で変化した．このことから，計算指向および GPU ベースのインスタンスタイプでは時間帯が運用上より重要な要因となることが示唆される．

4.3　時間帯が完了可能性に与える影響

図 4.2 に，$F_{\mathrm{comp}}(60)$ における時間帯間の変動が最も大きかった 4 つの組み合わせについて，$d \in \{5, 10, 30, 60\}$ 分に対する $F_{\mathrm{comp}}(d)$ を示す．

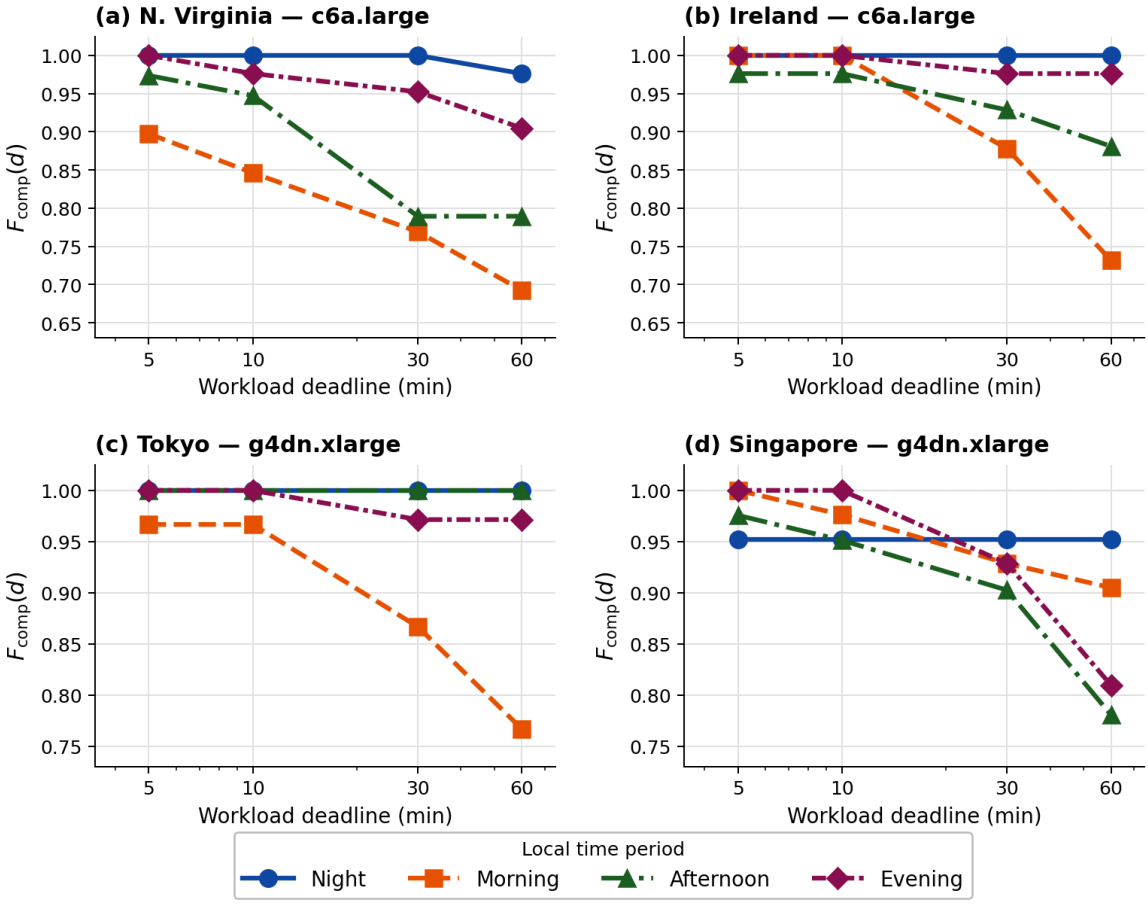


図 4.2　時間帯間の変動が最大であった 4 つのリージョン・インスタンスタイプの組み合わせにおける $F_{\mathrm{comp}}(d)$。線はローカル時間帯を表し，横軸は対数スケールである。

N. Virginia の c6a.large では，Night 期間に $F_{\mathrm{comp}}(60) = 0.98$（95% CI: 0.88–1.00; $n = 42$）を達成した一方，Morning 期間では $F_{\mathrm{comp}}(60) = 0.69$（95% CI: 0.54–0.81; $n = 39$）まで低下し，その差は約 0.29 であった．二つの信頼区間は重ならず，この差が標本変動に起因する可能性は低い．最も短い期限である $d = 5$ 分においても，Morning 期間では既に $F_{\mathrm{comp}}(5) = 0.90$（95% CI: 0.76–0.96）であり，中断リスクが起動後 5 分以内から蓄積し始めることを示している．

Ireland の c6a.large でも定性的に同様の傾向が見られた．Night 期間ではすべての期限において $F_{\mathrm{comp}}(60) = 1.00$（95% CI: 0.92–1.00; $n = 42$）を達成した一方，Morning 期間では 0.73（95% CI: 0.58–0.84; $n = 41$），Afternoon 期間では 0.88（95% CI: 0.75–0.95; $n = 42$）まで低下した．

Tokyo の g4dn.xlarge では Morning 期間に最大の低下が見られ，$F_{\mathrm{comp}}(60) = 0.77$（95%

CI: 0.59–0.88; $n = 30$）となった．Singapore では Afternoon 期間が最低の $F_{\mathrm{comp}}(60) = 0.78$（95% CI: 0.63–0.88; $n = 41$）を示した．

4 つの組み合わせ全体を通じて，Night 期間は一貫して最も高い，またはそれに近い $F_{\mathrm{comp}}(d)$ を示した．一方で最もリスクの高い時間帯はリージョンによって異なった．

4.4 考察

4.4.1 運用上のスケジューリング要因としての時間帯

結果は，計算指向および GPU ベースのインスタンスタイプにおいて $F_{\mathrm{comp}}(d)$ がローカル時間帯によって大きく変動する一方，t3.micro ではおおむね安定していることを示す．最大の差（約 0.29，信頼区間は非重複）は N. Virginia の c6a.large における Night と Morning の間で観測された．

観測された傾向は，ローカルなクラウド需要が日内周期に従うという仮説と整合する．ローカル夜間には業務利用に起因する計算および GPU 容量への需要が低下し，スポットインスタンスとして利用可能な余剰容量が増加すると考えられる．最もリスクの高い時間帯がリージョンによって異なることもこの解釈を支持する．N. Virginia と Ireland では Morning が最もリスクが高く，これは北米および欧州の業務開始時間帯と対応する．Singapore では Afternoon が最もリスクが高く，アジア地域の午後の利用ピークと整合する．

4.4.2 インスタンスタイプと中断されやすさ

t3.micro はすべての条件で $F_{\mathrm{comp}}(60) \geq 0.95$ を維持し，c6a.large および g4dn.xlarge と比較して高い安定性を示した．この違いは，各タイプに対するスポット市場での相対的需要に起因する可能性がある．バースト可能な軽量インスタンスは余剰容量をめぐる競合が小さい一方，計算最適化および GPU 搭載インスタンスは機械学習やバッチ処理の需要が高く，競合が激しい．この結果は Lee ら [8] の知見と整合する．

4.4.3 本章の限界

本章の評価にはいくつかの限界がある．

- 観測期間は 1 週間に限定されており，各時間帯のサンプル数はリージョン・タイプの組み合わせごとに約 30–42 である．結果は短期的変動を反映している可能性がある．
- 中断されない状態を完了可能性の proxy としており，実際のアプリケーションレベルのワークロードは実行していない．checkpointing，startup overhead，ジョブ間依存といったアプリケーション固有の要因は対象外である．
- 各組み合わせにつき 1 時間に 1 台のみ起動しているため，同時起動の影響やキュー挙動は捉えられていない．
- 最大スポット価格を指定していないため，デフォルト価格設定下の挙動のみを反映して

いる．

4.4.4 時間帯選択の適用限界

本章の結果は，実行時刻が有効な運用変数であることを示した．しかしこの手法には本質的な制約がある．

Night 期間に実行するためには，到着時刻によっては最大 24 時間の待機が必要となる．CI/CD の文脈において，プルリクエストに対するビルドやテストを夜間まで遅延させることは多くの場合受け入れられない．すなわち**時間帯の選択は，実行時刻を柔軟に調整できるワークロードにしか適用できない**．

この制約を解消する第二の運用変数を，次章で導入する．

第 5 章

分割粒度を考慮した実行モデル

5.1　本章のねらい

§1.3 で述べたとおり，本論文の問いは中断による無駄時間の最小化であり，無駄時間は式 (1.1) の二要素に分解される．

$$\text{無駄時間} \approx \underbrace{\text{やり直す回数}}_{\text{第 4 章で削減}} \times \underbrace{\text{1 回あたりの損失}}_{\textbf{本章で削減}}$$

第 4 章では第一要素を扱い，中断されにくい時間帯を選ぶことで $F_{\mathrm{comp}}(60)$ を 0.69 から 0.98 へ改善できることを示した．しかしこの手法は最大 24 時間の待機を要し，即時性が求められるワークロードには適用できない（§4.4.4）．

本章では**第二要素**，すなわち **1 回あたりの損失**の削減に取り組む．ジョブを k 個の断片に分割すれば，中断されても失われるのは 1 断片分で済む．分割は待機を必要としないため，実行時刻を選べないワークロードにも適用できる．

■既存データの再利用． 本章の分析には追加の測定を必要としない．第 3 章において $d \in \{5, 10, 30, 60\}$ を**異なるワークロードの実行時間要件**として測定したが，同じデータを**同一ジョブを k 分割したときの 1 断片の長さ**として読み替えれば，分割粒度の分析に直接利用できる．

■本章の構成． §5.2 でメモリレス性という前提を確認し，§5.3 で分割が効く理由を示す．§5.4 で第 3 章の指標を含む統合コストモデルを構築し，§5.5–5.7 でこれを実測データに適用する．§5.9 ではモデルの未確定要素を同定し，それを解消する検証計画を示す．

5.2　中断の近似的メモリレス性

分割が有効であるためには，中断の発生様式に関する前提が必要である．そこで，中断ハザード率が経過時間にどう依存するかを調べた．

経過時間区間ごとに，その区間に入る直前まで生存していたインスタンス数（リスク集合）で正規化したハザード率を表 5.1 および図 5.1 に示す．

ハザード率は 0.054–0.130 %/分の範囲で変動するが，**経過時間に対する単調増加の傾向は見**

表 5.1　経過時間別の中断ハザード率（$N_{\mathrm{succ}} = 1{,}950$，中断 117 件）

経過時間	中断件数	リスク集合	区間あたり	単位時間あたり
0–5 分	12	1,950	0.62%	0.123 %/分
5–10 分	11	1,938	0.57%	0.114 %/分
10–20 分	25	1,927	1.30%	0.130 %/分
20–30 分	12	1,902	0.63%	0.063 %/分
30–40 分	14	1,890	0.74%	0.074 %/分
40–50 分	23	1,876	1.23%	0.123 %/分
50–60 分	10	1,853	0.54%	0.054 %/分

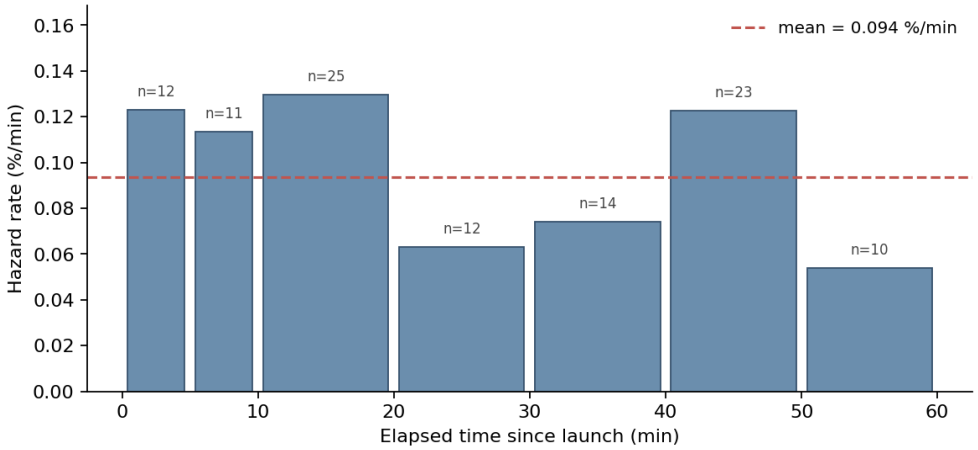


図 5.1　経過時間別の中断ハザード率。単調増加の傾向は見られず，0.054–0.130 %/分の範囲で変動する。

られない．すなわち「長時間稼働したインスタンスほど中断されやすくなる」という摩耗的な性質は観測されず，中断は単位時間あたりほぼ一定の確率で発生する**近似的にメモリレスな事象**として振る舞う．

■この主張の限界． 中断 117 件という標本数のもとでは，ハザードの定数性を統計的に強く主張することはできない．本論文では「単調増加の証拠が得られなかった」という弱い主張にとどめ，§5.9 の実験で追試する．

5.3　分割がもたらす二つの効果

メモリレス性のもとでは，ジョブを分割することにより二つの効果が同時に働く．

■効果 1：断片あたりの被中断確率の低下． 稼働時間が短いほど中断に遭遇する機会が減る．実測された F_{comp} を表 5.2 に示す．

■効果 2：中断時損失量の低下． 60 分のジョブが 55 分経過時点で中断されると 55 分が失われるが，10 分の断片であれば最大 10 分しか失われない（先行する断片はチェックポイント済みである）．中断された試行の平均稼働時間 $\bar{u}(T) = \mathbb{E}[U \mid U < T]$ も，T とともに減少する．

表 5.2　断片長 T に対する完了確率と中断時平均損失（全データをプール）

断片長 T	60 分	30 分	10 分	5 分
$F_{\mathrm{comp}}(T)$（点推定）	0.945	0.969	0.988	0.994
$\bar{u}(T)$（中断時損失）	26.4 分	12.7 分	4.9 分	2.3 分

これら二つが同時に効くため，分割の効果は非線形に大きい．

5.4　二変数を統合したコストモデル

5.4.1　記号と決定変数

ジョブ j は到着時刻 t_0 に到着し，次で特徴づけられる．

表 5.3　ジョブの特徴量

記号	名称	意味
W_j	総計算量	ジョブ全体の所要時間（分）
D_j	締切	$t_0 + D_j$ までに完了すべき
$k_{\max}$	分割上限	ジョブの性質により定まる分割可能数の上限

決定変数は**二つ組** (t, k) である．t は起動時刻（第 4 章の運用変数），k は分割数（本章で導入する運用変数）を表す．

分割数 k のとき，1 断片の長さは $T = W_j/k$ となる．

5.4.2　第 3 章の指標との接続

式 (1.1) で述べた無駄時間の二要素を，ここで具体的な量として書き下す．

■第一要素：やり直す回数． 断片の完了確率を $p(T, \pi)$ とする．ここで T は断片の長さ，$\pi = \pi(t)$ は時刻 t におけるローカル時間帯である（表 4.2）．

この p は，第 3 章で定義した期限条件付き完了可能性そのものである．

$$p(T, \pi) \;=\; F^{(\pi)}_{\mathrm{comp}}(T) \tag{5.1}$$

すなわち**第 3 章の指標が，本章のモデルの第一要素としてそのまま入力される**．成功するまで繰り返すときの期待試行回数は $1/p$ であるから，やり直す回数は $1/p - 1$ である．

■第二要素：1 回あたりの損失． 中断された試行で費やされた時間の期待値を

$$\bar{u}(T, \pi) \;=\; \mathbb{E}[\,U \mid U < T\,] \tag{5.2}$$

とする．U は観測された稼働時間であり，第 4 章のデータから直接算出できる．

■統合． 断片は k 個あり，各断片について起動オーバーヘッド s とチェックポイントコスト c を要する．したがって総無駄時間は次式で与えられる．

$$\boxed{\mathrm{Waste}(t,k) = \underbrace{k\left(\frac{1}{p(T,\pi(t))}-1\right)\bar{u}(T,\pi(t))}_{\text{やり直し}} + \underbrace{k\,(s+c)}_{\text{起動+チェックポイント}}} \tag{5.3}$$

ここで

- s：インスタンス起動オーバーヘッド。第 4 章のデータより**実測 4.52 秒**（中央値 4 秒，95 パーセンタイル 6 秒）。
- c：チェックポイントの保存および復元コスト。**本論文では未実測であり，§5.9 の測定対象とする。**

第 1 項は k の増加とともに減少し，第 2 項は k に比例して増加する．したがって Waste は k に対して U 字型となる．

5.4.3 最適化問題

以上より，提案手法は次の制約付き最適化問題として記述される．

$$\begin{aligned} \min_{t,\,k} \quad & \mathrm{Waste}(t,k) && \text{（総無駄時間）} \\ \text{s.t.} \quad & t \in [\,t_0,\; t_0 + D_j - W_j\,] && \text{（締切）} \\ & k \in \{1,2,\ldots,k_{\max}\} && \text{（分割可能性）} \\ & p(W_j/k,\,\pi(t)) \geq p_{\min} && \text{（信頼性下限）} \end{aligned} \tag{5.4}$$

探索空間は $24 \times k_{\max}$ 程度であり，全列挙により厳密に解ける．

5.4.4 第 3 章との関係

$k = 1$ に固定すると，式 (5.3) は

$$\mathrm{Waste}(t,1) = \left(\frac{1}{F_{\mathrm{comp}}(W_j)}-1\right)\bar{u} + s + c \tag{5.5}$$

となり，第 3 章で定義した F_{comp} に帰着する．すなわち**第 3 章のモデルは，本章のモデルにおいて $k=1$ とした特殊ケース**である．

5.5 最適分割数の存在

式 (5.3) を既存データに適用し，k に対する総無駄時間を算出した．推定には Wilson 95% 下側信頼限界を用い，標本数が少ないことによる楽観的推定を抑制した．

結果を表 5.4 に示す（$c = 0$ の場合）．

表 5.4　分割数 k に対する総無駄時間（$W_j = 60$ 分，$c = 0$，Wilson LCB）

k	断片長	p_{LCB}	やり直し	起動	**合計**	vs k=1
1	60.0 分	0.9341	1.86 分	0.08 分	**1.94 分**	—
2	30.0 分	0.9606	1.04 分	0.15 分	**1.19 分**	−39%
3	20.0 分	0.9675	0.97 分	0.23 分	**1.19 分**	−38%
4	15.0 分	0.9728	0.91 分	0.30 分	**1.21 分**	−38%
6	**10.0 分**	0.9824	0.53 分	0.45 分	**0.98 分**	**−50%**
10	6.0 分	0.9880	0.34 分	0.75 分	**1.09 分**	−44%
12	5.0 分	0.9893	0.30 分	0.90 分	**1.20 分**	−38%
20	3.0 分	0.9906	0.39 分	1.51 分	**1.90 分**	−2%
60	1.0 分	0.9980	0.00 分	4.52 分	**4.52 分**	+133%

$k = 6$（10 分ごとの分割）において総無駄時間は最小となり，$k = 1$ と比較して 50% の削減が得られる．一方 $k = 60$（1 分ごと）では起動コストが支配的となり，分割しない場合よりも 133% 悪化する．

すなわち**「細かく切るほど良い」という直観は誤りであり，最適分割数 k^* が存在する．**

5.6　チェックポイントコストに対する感度

表 5.4 は $c = 0$ を仮定した理想値である．実際にはチェックポイントの保存と復元に時間を要する．c を変化させたときの k^* を表 5.5 および図 5.2 に示す．

表 5.5　チェックポイントコスト c に対する最適分割数 k^* の感度。各セルは総無駄時間（分）を表し，太字は各行の最小値を示す。

c (秒)	k=1	k=2	k=4	k=6	k=12	$\mathbf{k^*}$
0	1.94	1.19	1.21	**0.98**	1.20	**6**
5	2.02	**1.36**	1.54	1.48	2.20	**2**
10	2.11	**1.52**	1.88	1.98	3.20	**2**
30	2.44	**2.19**	3.21	3.98	7.20	**2**
60	**2.94**	3.19	5.21	6.98	13.20	**1**
120	**3.94**	5.19	9.21	12.98	25.20	**1**

この結果は本研究にとって決定的である．

- わずか 5 秒のチェックポイントコストで k^* は 6 から 2 へ低下する．
- $c \geq 60$ 秒では $k^* = 1$ となり，**分割そのものが無効**となる．

すなわち**提案手法が成立するか否かは c の値によって決まる**．c は本論文では未実測であり，§5.9 における最優先の測定対象となる．

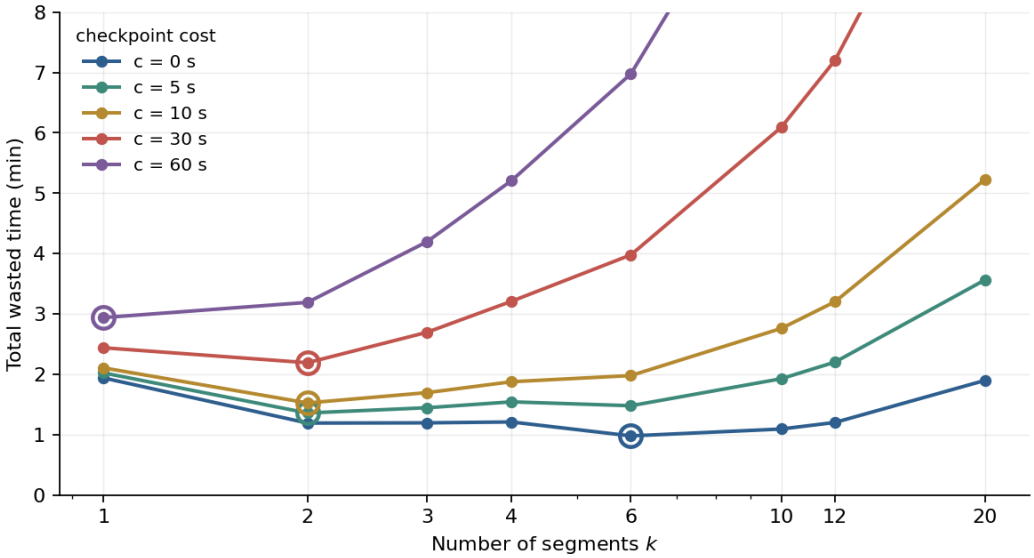


図 5.2　分割数に対する総無駄時間の U 字カーブ。各曲線はチェックポイントコスト c の値に対応し，二重丸は各曲線の最小点 k^* を示す。c の増加とともに k^* は左へ移動する。

5.7　実行時刻との併用

実行時刻（第 4 章）と分割粒度（本章）は独立した運用変数である．両者を組み合わせた場合の総無駄時間を表 5.6 および図 5.3 に示す．

表 5.6　ローカル時間帯と分割数の組み合わせによる総無駄時間（分，$c = 0$，Wilson LCB）

時間帯	k=1	k=2	k=6	k=12
Night	1.09	0.73	**0.60**	1.19
Morning	**3.81**	2.13	*1.62*	1.72
Afternoon	2.58	2.18	1.69	1.53
Evening	2.05	1.00	1.07	0.90

表 5.6 から次の三点が読み取れる．

1. 最悪条件（Morning，k=1）の 3.81 分に対し，最良条件（Night，k=6）は 0.60 分であり，84% の削減となる．
2. **実行時刻を選べない場合でも**，Morning において k=6 とすれば 3.81 分から 1.62 分へ 57% 削減される．これは時刻選択に匹敵する効果を**待機時間ゼロ**で得られることを意味する．
3. 両者は独立に作用するため，併用により最大の効果が得られる．

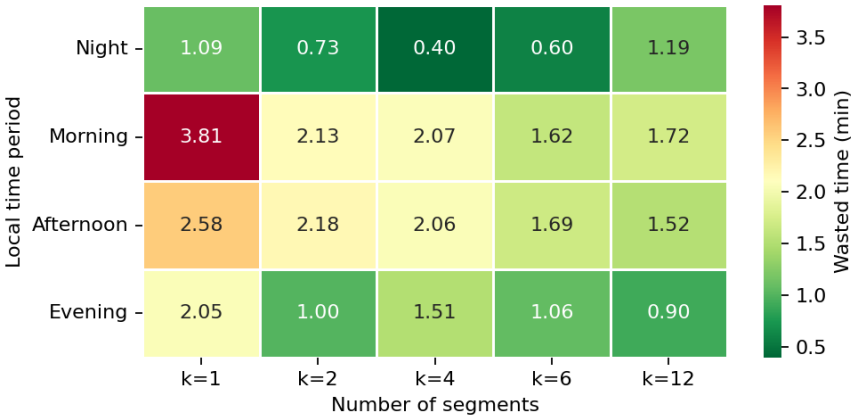


図 5.3　ローカル時間帯と分割数の組み合わせによる総無駄時間。色が濃いほど無駄時間が大きい。

5.8　比較対象とする戦略

§5.9 の実験では，式 (5.4) の制約を変化させることで定義される以下の戦略を比較する．

表 5.7　比較する戦略

戦略	(t, k)	位置づけ
B0: Naive	t_0 固定，k=1	素朴な実行（ベースライン）
B1: 時刻のみ	t 自由，k=1	**第 4 章の手法**
B2: 分割のみ	t_0 固定，k 自由	待機時間ゼロ
B3: 併用（提案）	t 自由，k 自由	完全版

B1 と B2 を分離することにより，実行時刻と分割粒度それぞれの寄与を独立に評価できる．

5.9　モデルの未確定要素と検証計画

§5.6 で示したとおり，本章のモデルには一つの未確定要素がある。チェックポイントコスト c である。c の値によって k^* は 6 から 1 まで変化し，$c \geq 60$ 秒では分割そのものが無効となる。

すなわち**本章の提案が成立するか否かは c に依存する**。s が第 4 章のデータから実測できたのに対し，c はワークロードの性質に依存するため，既存データからは算出できない。

本節では，この未確定要素を解消し，あわせてモデルの妥当性を検証する計画を示す。

5.9.1　実験の目的と優先順位

§5.6 で示したとおり，提案手法の成否はチェックポイントコスト c に依存する．したがって実験の目的を以下の優先順位で設定する．

表 5.8　実験の目的と優先順位

優先	目的	理由
1	c **の実測**	k^* を決定する最重要パラメータであり未知。$c \geq 60$ 秒であれば提案手法の前提が崩れる
2	U 字カーブの検証	式 (5.3) の妥当性検証
3	中断の独立性の検証	期待試行回数 $1/p$ の前提の検証
4	実行時刻との併用効果	第 4 章との接続

5.9.2　対象ワークロードの選定

5.9.3　選定基準

- 実行時間が 30 分以上であること（短いと $F_{\text{comp}} > 0.99$ となり差が検出できない）
- チェックポイントが実装可能であること
- c が現実的に測定できること
- 中断率の高い条件を選択できること

5.9.4　候補の比較

c の大きさが対照的な二つのワークロードを選定する．

表 5.9　対象ワークロードの候補

	A: 機械学習の学習	**B: CI/CD テスト**
分割の単位	エポック	テストシャード
チェックポイント	モデルの保存・復元	結果 JSON の保存
想定 c	**30–120 秒**（数百 MB）	**1–5 秒**（数 KB）
予測 k^*	**1–2（分割は無効）**	**2–6（分割が有効）**
インスタンス	g4dn.xlarge	c6a.large
中断率	高い	低い
既発表との接続	弱い	**強い**

c が対照的であるため，両者を実施することで**どのようなワークロードであれば分割が有効か**という一般的な指針が得られる．これは単一ワークロードによる評価よりも強い主張となる．

一方のみを実施する場合は B（CI/CD テスト）を選択する．第 4 章との接続が強く，コストが約 1/10 であり，c が小さく分割が有効となる可能性が高いためである．

5.9.5 実験 1：チェックポイントコストの実測

■目的． c を実測し，図 5.2 の理論曲線上で k^* を確定する．

■方法． 中断のない環境において分割数 k を変化させ，純粋なオーバーヘッドのみを測定する．各断片についてチェックポイントの保存時間，復元時間，インスタンス起動時間を記録し，$c =$ 保存 $+$ 復元 の平均を求める．

■規模． $k \in \{1, 2, 4, 6, 12\}$ をそれぞれ 10 回実施する．断片数の合計は $(1{+}2{+}4{+}6{+}12){\times}10 = 250$ 本，総計算時間は約 50 時間である．中断の発生を待つ必要がないため **1–2 日で完了**する．

■意義． この実験の結果のみで，理論曲線上の k^* が確定する．以降の実験設計はこの結果を踏まえて確定させる．

5.9.6 実験 2：U 字カーブの検証

■目的． 実測された c のもとで予測される k^* が，実際に総無駄時間を最小化するかを検証する．

■方法． 中断が発生しやすい条件として N. Virginia, c6a.large, Morning 期間を選択する（第 4 章において $F_{\mathrm{comp}}(60) = 0.69$ と最低値を記録）．$k \in \{1, 2, 4, 6, 12\}$ を各 24 回／日，14 日間実施する．中断された場合は当該断片のみを再実行し，先行する断片はチェックポイントから復元する．

■測定項目． 総課金時間，やり直し回数，完了までの実時間．

■規模． $5 \times 24 \times 14 = 1{,}680$ 本の実行．

5.9.7 実験 3：中断の独立性の検証

■目的．「中断された直後の再試行も同じ確率 p で成功する」という仮定を検証する．

式 (5.3) における期待試行回数 $1/p$ は，各試行が独立であることを前提としている．しかし中断は容量逼迫時に発生すると考えられるため，直後の再試行はより失敗しやすい可能性がある．その場合 p は過大評価であり，分割の効果も割り引いて解釈する必要がある．

■方法． 中断を検知した場合に即座に同一条件で再起動し，2 回目の成否を記録する．1 回目の p と 2 回目の p を比較する．

■規模． 中断 50 件の収集を目標とする．中断率 6% のもとでは約 830 回の起動を要する．**実験 2 と同一の実行に相乗りできる**（実験 2 で中断された断片の再試行結果を記録すればよい）．

5.9.8 実験 4：実行時刻との併用

■目的． 第 4 章の手法（実行時刻）と第 5 章の手法（分割粒度）の寄与を分離して評価する．

■方法． 表 5.7 の B0–B3 を比較する．表 5.6 に基づく予測値は 3.81 / 1.09 / 1.62 / 0.60 分である．

■期待される知見． B2（分割のみ）が B1（時刻のみ）に匹敵すれば，**待機を伴わずに同等の効果が得られる**ことが実証される．

5.9.9 実験規模のまとめ

表 5.10　実験の規模とコスト（c6a.large を想定）

実験	期間	コスト	得られるもの
1: c の実測	**1–2 日**	\$1.5	k^* **の確定**
2: U 字検証	2 週間	\$25	モデルの妥当性
3: 独立性	実験 2 に相乗り	\$0	前提の検証
4: 併用	2 週間	\$25	第 4 章との接続
合計	**約 1 か月**	**約 \$52**	

機械学習ワークロード（実験 1 のみ g4dn.xlarge で追加実施）を含めても約 \$70 である．

5.9.10 実装方針

第 4 章で構築した測定フレームワークを流用する．変更点を表 5.11 に示す．

表 5.11　既存実装からの変更点

既存要素	変更内容
起動 Lambda	user-data の待機処理を実ワークロードに置換
中断検知ループ	そのまま流用（インスタンスメタデータの監視）
記録テーブル	断片番号，分割数，保存時間，復元時間を追加
スケジューラ	「断片完了後に次の断片を起動」する連鎖を追加

チェックポイントの保存先はオブジェクトストレージとし，各断片は直前の断片が保存した状態を読み込み，処理後に次の状態を書き出す．

5.9.11 想定される結果

実験 1 の結果に応じて，導かれる結論は以下のように分岐する．**いずれの場合も報告価値のある結果となる．**

表 5.12　チェックポイントコストの実測値に応じた結論

c の実測値	k^*	導かれる結論
$c < 5$ 秒	4–6	分割は実行時刻の選択を代替できる。最適粒度は約 10 分
$5 \le c < 30$ 秒	2	分割は有効だが粗く切るべきであり，細分化は逆効果
$30 \le c < 60$ 秒	2	分割による利得は限定的
$c \ge 60$ 秒	1	**分割は成立しない。実行時刻の選択が唯一の手段であり，第 4 章の手法の価値が補強される**

5.9.12　リスクと対策

表 5.13　想定されるリスクと対策

リスク	影響	対策
c が大きく分割が無効	提案の前提が崩れる	実験 1 を最優先し 1–2 日で判明させる。無効でも否定的結果として報告する
中断が十分に集まらない	実験 2 の検出力不足	中断率が最も高い条件に集中させる
ハザード一定が偽	メモリレス性の前提が崩れる	実験で追試する。標本数 117 の限界を明記済み（§5.2）
独立性が成立しない	p が過大評価となる	実験 3 で検証し，必要なら条件付き確率へ修正する
観測期間と実験期間の市場変化	予測が外れる	実験期間中に F_{comp} を再測定し比較する

5.10　補足：リージョン選択について

第 4 章の考察では，実行時刻を調整できない場合の代替としてリージョン選択に言及した．しかし既存データを用いた検討の結果，**コスト最小化を目的とする限りリージョン選択は機能しない**ことが判明した．

表 5.14 に示すとおり，リージョン間の価格差（34–41%）が信頼性の差（期待課金時間にして数パーセント）を大きく上回る．このため常に最安リージョンが選択され，リージョンを自由に選べる戦略はリージョンを固定した戦略と同一の結果となった．

この否定的結果は，実行時刻と分割粒度という二つの運用変数に注力する本研究の方針を支持するものである．

表 5.14　c6a.large のリージョン別観測スポット価格

リージョン	価格 (USD/h)	N. Virginia 比
N. Virginia	0.0295	—
Singapore	0.0394	+34%
Ireland	0.0400	+36%
Tokyo	0.0417	+41%

第 6 章

結論

6.1 本論文のまとめ

本論文は，**中断によって失われる時間を最小化するにはジョブをどう実行すればよいか**という問いに取り組んだ．

この無駄時間が**やり直す回数**と **1 回あたりの損失**の積に分解されること（式 (1.1)）に着目し，前者を**実行時刻**，後者を**分割粒度**という二つの運用変数によって制御する枠組みを構築した．

第 3 章では，スポットインスタンスの信頼性を起動可能性ではなく**期限条件付き完了可能性** $F_{\mathrm{comp}}(d)$ として再定義し，起動成功と起動後の中断挙動を分離して記録する測定フレームワークを示した．

第 4 章では，AWS の 4 リージョン・3 インスタンスタイプを対象とした 1 週間の実測（$N_{\mathrm{req}} = 2{,}016$）により，$F_{\mathrm{comp}}(d)$ がローカル時間帯によって大きく変動することを示した．計算指向インスタンスでは Night 期間の 0.98 に対し Morning 期間では 0.69 まで低下し，**実行時刻**が運用上意味のある変数であることを明らかにした．一方で，この手法は最大 24 時間の待機を要するという制約を持つ．

第 5 章では，同一データの再分析により中断ハザード率が経過時間によらずほぼ一定であることを示し，この近似的メモリレス性に基づいて第二の運用変数である**分割粒度**を導入した．そして第 3 章の指標 F_{comp} を第一要素，分割粒度 k を第二要素として組み込んだ**二変数統合コストモデル**を構築した（式 (5.3)）．このモデルにおいて第 4 章の手法は $k = 1$ の特殊ケースに対応する．

モデルを実測データに適用した結果，総無駄時間が k に対して U 字型となること，すなわち**最適分割数 k^* が存在する**ことを示した．さらに感度分析により，k^* が**チェックポイントコスト c に支配される**こと，$c \geq 60$ 秒では分割が無効となることを明らかにした．また二変数が独立に作用するため併用可能であり，実行時刻を選べない場合でも分割のみで時刻選択に匹敵する効果が得られることを示した．

さらに §5.9 では，モデルの未確定要素である c を同定し，その実測を最優先とする検証計画を示した．

6.2 本研究の意義

本研究の意義は以下の三点である．

1. スポットインスタンスの信頼性を，プロバイダ固有の不透明な指標ではなく，直接観測された起動・中断記録のみから算出される**解釈可能な指標**として定式化した．
2. 実行時刻という既知の変数に加え，**分割粒度**という新たな運用変数を導入し，待機を伴わない中断対策の可能性を示した．
3.「細かく分割するほど良い」という直観に反し，**最適分割数が存在し，それがチェックポイントコストに支配される**ことを定量的に示した．

6.3 今後の課題

- §5.9 の検証計画の実施，とりわけチェックポイントコスト c の実測
- 観測期間の延長による，季節変動を含む長期的傾向の把握
- 中断のメモリレス性に関する，より大きな標本に基づく検証
- SPS など既存のプロバイダ固有指標との定量的比較
- 実際の CI/CD ワークロードおよび機械学習ワークロードにおける評価

謝辞

本研究を進めるにあたり，終始丁寧なご指導を賜りました指導教員の先生方に深く感謝いたします．また，日頃より有益な議論をいただいた研究室の皆様に御礼申し上げます．

参考文献

[1] Amazon Web Services: Amazon EC2 Spot Instances. https://aws.amazon.com/ec2/spot/ (Accessed: May 2026)

[2] Lin, L., Pan, L., Liu, S.: Methods for improving the availability of spot instances: A survey. Computers in Industry, 141, 103718 (2022).

[3] Ramesh, D.G.A., Lennon, R.G.: Evaluating the Impact of Reserved Instances vs. Spot Instances in DevOps Cloud Workflows. In: 2025 35th Irish Signals and Systems Conference (ISSC). IEEE (2025).

[4] Garg, D., Kaata, S.K., Bhumireddy, D.R., Bolla, B.K.: Reducing Cloud Cost Through Optimized Bidding for AWS Spot Instances. In: 2025 3rd International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT). IEEE (2025).

[5] Katayama, D., Kasai, K., Koita, T.: Migration Destination Selection Algorithm for Spot Instances using SPS. In: 2022 IEEE International Conference on Big Data (Big Data), pp. 6690–6692. IEEE, Osaka (2022).

[6] Kim, K., Lee, K.: Making Cloud Spot Instance Interruption Events Visible. In: Proceedings of the ACM Web Conference 2024 (WWW '24), pp. 2998–3009. ACM, New York (2024).

[7] Amazon Web Services: Spot Placement Score. Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-placement-score.html (Accessed: May 2026)

[8] Lee, S., Hwang, J., Lee, K.: SpotLake: Diverse Spot Instance Dataset Archive Service. In: 2022 IEEE International Symposium on Workload Characterization (IISWC), pp. 242–255. IEEE (2022).

[9] Pham, T.-P., Ristov, S., Fahringer, T.: Performance and Behavior Characterization of Amazon EC2 Spot Instances. In: 2018 IEEE 11th International Conference on Cloud Computing (CLOUD). IEEE (2018).

[10] Amazon Web Services: Amazon EC2 Instance Types. Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-types.html (Accessed: May 2026)